DÉMO 003-E · CHANGEMENTS ENTRE LES ÉTAPES

Événements

Chaque étape comparée à la précédente : ce qui est ajouté et ce qui est retiré.

`+ ligne ajoutée` `– ligne retirée` `@@ position dans le fichier`

1. **Étape 1** · etape-1-oncopy (3 fichiers, en entier)
2. **Étape 1 → 2** · etape-2-event-target (1 fichier)
3. **Étape 2 → 3** · etape-3-state (1 fichier)
4. **Étape 3 → 4** · etape-4-classname (2 fichiers)
5. **Étape 4 → 5** · etape-5-onchange (2 fichiers)

Étape 1 etape-1-oncopy : point de départ

onCopy + alert (camelCase)

Le code de départ, en entier : les fichiers écrits pour la démo et utilisés (atteints par les imports depuis src/main.*) ; les fichiers du gabarit Vite non modifiés sont seulement listés. Ordre de lecture : main → App → composants et pages → le reste.

Styles fournis (copiés tels quels, pas tapés en live) : src/App.css (42 lignes)

Fichiers générés par Vite, non modifiés (non affichés) : index.html, vite.config.js, .gitignore, eslint.config.js, public/vite.svg

package.json (gabarit Vite) : dépendances ajoutées : react-slick ^0.31.0

DÉPART **src/main.jsx** 6 lignes

```jsx
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <App />
);
```

DÉPART **src/App.jsx** 8 lignes

```jsx
import "./App.css";
import Title from "./components/Title.jsx";

function App() {
  return <Title />;
}

export default App;
```

DÉPART **src/components/Title.jsx** 10 lignes

```jsx
// Démo 003-E · étape 1 : onCopy + alert (les événements React sont en camelCase : onCopy, pas oncopy)
function Title() {
  return (
    <h1 onCopy={() => alert("Texte copié !")}>
      Bienvenue dans la formation React
    </h1>
  );
}

export default Title;
```

Étape 1 → 2 etape-1-oncopy → etape-2-event-target

onMouseEnter / onMouseLeave : event.target.style.color = "red" (ça marche, mais ce n'est pas la bonne façon)

MODIFIÉ **src/components/Title.jsx** +14 −2

```
@@ -1,7 +1,19 @@
-// Démo 003-E · étape 1 : onCopy + alert (les événements React sont en camelCase : onCopy, pas oncopy)
+// Démo 003-E · étape 2 : onMouseEnter / onMouseLeave, on change la couleur directement avec l'événement
+// Ça marche, mais ce n'est pas la bonne façon : on modifie le DOM à la main
 function Title() {
+  const handleMouseEnter = (event) => {
+    event.target.style.color = "red";
+  };
+  const handleMouseLeave = (event) => {
+    event.target.style.color = "black";
+  };
+
   return (
-    <h1 onCopy={() => alert("Texte copié !")}>
+    <h1
+      onCopy={() => alert("Texte copié !")}
+      onMouseEnter={handleMouseEnter}
+      onMouseLeave={handleMouseLeave}
+    >
       Bienvenue dans la formation React
     </h1>
   );
```

Étape 2 → 3 etape-2-event-target → etape-3-state

Un state isHover : React gère la couleur (+ transition)

MODIFIÉ **src/components/Title.jsx** +6 −10

```
@@ -1,18 +1,14 @@
- // Démo 003-E · étape 2 : onMouseEnter / onMouseLeave, on change la couleur directement avec l'événement
- // Ça marche, mais ce n'est pas la bonne façon : on modifie le DOM à la main
+ import { useState } from "react";
+ // Démo 003-E · étape 3 : la bonne façon, un state isHover ; React gère l'affichage
  function Title() {
-   const handleMouseEnter = (event) => {
-     event.target.style.color = "red";
-   };
-   const handleMouseLeave = (event) => {
-     event.target.style.color = "black";
-   };
+   const [isHover, setIsHover] = useState(false);

    return (
      <h1
        onCopy={() => alert("Texte copié !")}
-       onMouseEnter={handleMouseEnter}
-       onMouseLeave={handleMouseLeave}
+       onMouseEnter={() => setIsHover(true)}
+       onMouseLeave={() => setIsHover(false)}
+       style={{ color: isHover ? "red" : "black", transition: "all 0.3s ease" }}
      >
        Bienvenue dans la formation React
      </h1>
```

Étape 3 → 4 etape-3-state → etape-4-classname

Template string avec interpolation pour choisir la classe : title--hover ou title--normal (fichier Title.css)

AJOUTÉ `src/components/Title.css` +13 −0

```
@@ -1,0 +1,13 @@
+ .title {
+   padding: 20px;
+   transition: all 0.3s ease;
+ }
+
+ .title--normal {
+   color: black;
+ }
+
+ .title--hover {
+   color: red;
+   transform: scale(1.05);
+ }
```

MODIFIÉ `src/components/Title.jsx` +3 −2

```
@@ -1,5 +1,6 @@
  import { useState } from "react";
- // Démo 003-E · étape 3 : la bonne façon, un state isHover ; React gère l'affichage
+ import "./Title.css";
+ // Démo 003-E · étape 4 : une template string avec interpolation pour choisir la classe CSS
  function Title() {
    const [isHover, setIsHover] = useState(false);

@@ -8,7 +9,7 @@
        onCopy={() => alert("Texte copié !")}
        onMouseEnter={() => setIsHover(true)}
        onMouseLeave={() => setIsHover(false)}
-       style={{ color: isHover ? "red" : "black", transition: "all 0.3s ease" }}
+       className={`title ${isHover ? "title--hover" : "title--normal"}`}
      >
        Bienvenue dans la formation React
      </h1>
```

Étape 4 → 5 etape-4-classname → etape-5-onchange

onChange : on tape un nom → « Bonjour, Samir » ; champ vide → « Bonjour, invité ». Le CSS du champ est déjà dans App.css depuis l'étape 1

MODIFIÉ `src/App.jsx` +7 −1

```
@@ -1,8 +1,14 @@
  import "./App.css";
  import Title from "./components/Title.jsx";
+ import Greeting from "./components/Greeting.jsx";

  function App() {
-   return <Title />;
+   return (
+     <>
+       <Title />
+       <Greeting />
+     </>
+   );
  }

  export default App;
```

AJOUTÉ `src/components/Greeting.jsx` +18 −0

```
@@ -1,0 +1,18 @@
+ import { useState } from "react";
+ // Démo 003-E · étape 5 : onChange, l'utilisateur tape, on met à jour le state, React met à jour l'affichage
+ function Greeting() {
+   const [name, setName] = useState("");
+
+   return (
+     <div>
+       <input
+         placeholder="Votre nom"
+         value={name}
+         onChange={(e) => setName(e.target.value)}
+       />
+       <p>Bonjour, {name || "invité"}</p>
+     </div>
+   );
+ }
+
+ export default Greeting;
```